Full POS transaction verification

Application update, 2026-09-24 04:42 UTC: the separately user-authorized application posting-cycle migration is now complete and verified. The specific schema blocker described in the earlier snapshots below is cleared. See the final application rollout addendum and `application-migration-result.json`; earlier observations are retained as historical evidence.

Status: **PASS for two bounded full-kernel cases on the approved disposable schema; application release remains BLOCKED by confirmed schema drift.** Recorded 2026-09-24. The two new tests extend the earlier 19 bounded PostgreSQL cases to **21 distinct passing cases**. Their original evidence limitations remain applicable.

High-severity runtime and release blocker

Current `services/accounting/posting.service.ts:187` uses the generated `LedgerPostingBatch` model, which selects `postingCycle`. The column remains absent in the local application database. The initial full POS test reached real stock movement, payment creation, receivable document/state and customer balance work, then accounting failed with Prisma P2022. That initial failure was not accepted as the intended late rollback proof. The approved disposable-only schema correction below enabled the full suite to pass; it did not repair the application database.

Read-only metadata inspection of **30 POS-dependent models**, repeated after the approved disposable correction, confirmed:

- Local application database `localhost:5432/stoquify_replay_next_20260830`: `ledger_posting_batches.postingCycle` remains absent.
- Recorded disposable test database `localhost:5432/stoquify_pos_audit_20260923_d1532fb1`: no missing scalar/enum columns among those 30 models.

No other missing scalar/enum columns were found among those 30 generated models. This check does not certify indexes, constraints, enum values, every model, data consistency or migration history. Evidence: `pos-schema-column-audit.json`, generated by `audit-pos-schema.cjs` using `BEGIN READ ONLY` and information_schema queries only. Application business records were neither read nor changed.

Exact approved disposable-only migration

Existing repository file: `prisma/migrations/20260909093000_add_suspense_posting_cycles/migration.sql`.

SHA256: `CD5D20A342504C2F1D27C8644C3D41A8F80019D56A0A459678166DF6FF8F119F`.

The user explicitly approved **“Approve the disposable-database migration”** for **only that exact migration** against the recorded disposable database `localhost:5432/stoquify_pos_audit_20260923_d1532fb1`. A dedicated one-purpose runner verified the approved SHA, recorded target and `current_database()` before executing it. The application database was explicitly excluded and not connected by this runner. Application remediation still needs its own reviewed rollout.

Reviewed SQL effects:

1. Begins one transaction; applies a 5-second lock timeout and 60-second statement timeout.

2. Adds `ledger_posting_batches.postingCycle INTEGER NOT NULL DEFAULT 1`, preserving existing rows as cycle 1.
3. Adds a check permitting cycles above 1 only for PAYMENT_SUSPENSE with SUSPENSE_RECLASSIFICATION or REVERSAL purpose.
4. Creates unique index `ledger_posting_batches_source_cycle_key` on organization, source type, source ID, posting purpose and cycle.
5. Drops the old unique index `ledger_posting_batches_organizationId_sourceType_sourceId_p_key` only after the replacement exists, then commits.

No row deletion, provider call, live charge/refund, country pack change or broad migration deployment occurred. The SQL changes constraints and the uniqueness definition. It is not idempotent: the dedicated runner refuses to reapply it if the column already exists.

Automatic approval review initially rejected making migration application incidental to the reusable test runner: **"The patch would make the test runner apply a schema migration as incidental verification; the user's brief explicitly prohibits broad migrations, even though the target is disposable."** That rejected patch did not land or alter either database. Work resumed only after the explicit approval above, using `apply-approved-posting-cycle-disposable.cjs`. The reusable test runner remains migration-free.

Migration completed at `2026-09-24T03:26:01.469Z`, exit 0. Retained artifacts: exact SQL `postgres-approved-posting-cycle.sql`; authorization/target/hash/timestamps/exit and resulting column verification `postgres-approved-posting-cycle.json`.

Test implementation and evidence boundaries

`services/pos/__tests__/pos-full-commit.postgres.test.ts` now contains two opt-in tests, guarded by `RUN_POS_FULL_COMMIT_POSTGRES=1`, exact local disposable database name validation and a read-only column prerequisite check:

- Concurrent identical requests commit one tracked-stock sale, cash drawer inflow, on-account customer balance, immutable receivable/state, balanced sale/payment journals, business events/outbox and durable result; one result is original and one a replay. A later replay leaves the full financial snapshot unchanged.
- A tenant-scoped trigger fails the final COMMITTED registry transition after all real kernels run. A temporary nontransactional sequence probe proves that exact late boundary was reached. The test verifies rollback of stock, cash, customer balance, payments, journals, posting batches, receivables/states, events/outbox and audits, preservation of the retained request, and successful retry after removing only the owned test trigger/function/sequence.

The synthetic fixture uses tracked inventory, CASH plus ON_ACCOUNT tender, an open period, leaf accounts, active sales/cash journals and balanced zero-tax posting rules. It is not a statutory chart, tax/country validation or production configuration recommendation. No internal stock/accounting/customer/event collaborator is mocked; the suite asserts the key functions are real. Cash/on-account requires no external payment provider or delivery simulation. Fiscalization remains queued and nonauthoritative.

Current execution evidence:

Check	Outcome
Full integration after approved disposable migration	**PASS**, exit 0, 1 suite / 2 tests. `services-postgres-full-commit-final.log` and `services-postgres-exits-full-commit.json`.
Late finalization-failure probe	**PASS**: exact final boundary fired, all financial snapshots rolled back, retained request survived, same request retried once successfully.
Existing 19 PostgreSQL cases	Remain the previously recorded passing bounded cases; combined distinct count is 21.
Final new-test ESLint	**PASS**, exit 0; `services-full-commit-eslint-final.log` (empty).
Read-only schema audit	**PASS**, exit 0; application column still absent, disposable corrected.
Exact approved migration application	**PASS**, exit 0, only the approved disposable target.

Exact commands, run from the repository root, all exit 0:

```
node what-next/pos-enterprise-modernization/2026-09-23/evidence/apply-approved-posting-cycle-disposable.cjs
node what-next/pos-enterprise-modernization/2026-09-23/evidence/run-pos-postgres-existing.cjs full-commit
node what-next/pos-enterprise-modernization/2026-09-23/evidence/audit-pos-schema.cjs
node node_modules/eslint/bin/eslint.js services/pos/__tests__/pos-full-commit.postgres.test.ts
```

The test runner sets `NODE_ENV=test`, `TS_NODE_TRANSPILE_ONLY=true`, `TS_NODE_COMPILER_OPTIONS={"moduleResolution":"node"}`, `RUN_POS_FULL_COMMIT_POSTGRES=1` and the exact recorded disposable connection. It invokes `node node_modules/jest/bin/jest.js --runInBand --roots services/pos/__tests__ --runTestsByPath services/pos/__tests__/pos-full-commit.postgres.test.ts`; it does not apply migrations.

Full-suite run: `2026-09-24T03:27:36.418Z` through `2026-09-24T03:28:57.773Z`; Jest reported 54.393 seconds. Expected diagnostic logs contain the concurrent write-conflict retry and deliberately injected final-evidence failure; both cases pass. No financial boundary assertions were removed or replaced with internal mocks to obtain this result.

Application migration preflight addendum

Metadata-only inspection at `2026-09-24T04:04:29.457Z` of the exact recorded application target `localhost:5432/stoquify_replay_next_20260830` completed with exit 0, inside a verified `BEGIN READ ONLY` transaction. Evidence: `app-posting-cycle-preflight.json`, including the actual catalog queries, migration SHA, migration-history metadata, column definitions, indexes and constraints. No business rows were read and no database mutation occurred.

Prisma's `public._prisma_migrations` has **no row** for `20260909093000_add_suspense_posting_cycles`; its overall metadata contains 86 finished migrations and zero unfinished migrations. The physical table matches an **unapplied** posting-cycle migration: `postingCycle`, the cycle-scope check and replacement cycle unique index are absent. The old unique index on organization/source type/source ID/posting purpose is present, unique, valid and ready; the separate idempotency-key unique index is also valid and ready. History does not falsely claim this migration is already applied.

The actionable next step is a separately authorized application rollout of this exact, already reviewed migration with its recorded SHA, retaining execution evidence and keeping Prisma history consistent with the successfully verified schema. Do not mark the migration applied before its SQL succeeds or run an unrestricted pending-migration deployment for this single fix. After that rollout, verify the new column/default/check/index, removal of the old index, and the matching completed migration-history entry before clearing the runtime blocker. This preflight did not apply or authorize that rollout and does not certify unrelated pending migrations or application data.

Approved application rollout completed

The user subsequently explicitly requested completion of the application's remaining `postingCycle` migration. The guarded runner `application-migration-posting-cycle.cjs` revalidated **only** `localhost:5432/stoquify_replay_next_20260830`, confirmed the current unapplied schema/history, and preserved table column/index/constraint and migration-history metadata before mutation. The repository migration's SHA remained `cd5d20a342504c2f1d27c8644c3d41a8f80019d56a0a459678166df6ff8f119f`.

The exact existing transactional SQL committed at `2026-09-24T04:42:04.080Z`. After verifying the resulting schema, the installed Prisma 6.19.3 CLI reconciled this one migration through its supported `migrate resolve --applied` command. Final verification completed at `2026-09-24T04:42:12.194Z`, **exit 0**.

Verified results:

- `postingCycle` is INTEGER NOT NULL with default 1.
- The cycle-scope check is validated; the new cycle unique index is unique, valid and ready; the old source unique index is absent.
- The separate idempotency unique index and all prior column/constraint metadata are unchanged.
- Exactly one completed migration-history row has the approved SHA and no rollback timestamp. The original 86 migration records are unchanged; history now has 87 records.
- Prisma reports `applied_steps_count: 0` for supported history reconciliation after independently executed SQL. The separately retained SQL execution and verified schema provide execution evidence; no history field was manually fabricated.

Exact commands (all exit 0):

```
node --check what-next/pos-enterprise-modernization/2026-09-23/evidence/application-migration-posting-cycle.cjs
node what-next/pos-enterprise-modernization/2026-09-23/evidence/application-migration-posting-cycle.cjs --preflight
node what-next/pos-enterprise-modernization/2026-09-23/evidence/application-migration-posting-cycle.cjs --apply-approved
```

The application runner invoked only this history command, after SQL and schema verification:

```
node node_modules/prisma/build/index.js migrate resolve --config prisma.config.ts --schema prisma/schema.prisma --applied 20260909093000_add_suspense_posting_cycles
```

Evidence: `application-migration-preflight.json`, `application-migration-before.json`, `application-migration-approved.sql`, `application-migration-after-sql.json`, `application-migration-resolve.log`,

`application-migration-after.json`, and `application-migration-result.json`. The runner refuses reapplication when the column/history or execution evidence already exists, so it is not a general-purpose migration launcher.

This closes the specific application's `ledger_posting_batches.postingCycle` schema/runtime blocker. No sales, refunds, voids, access changes, unrelated migrations or production deployment were performed. The previous 21 PostgreSQL verification cases retain their documented scopes; this rollout does not expand them into statutory/provider or complete release certification. Live localized cashier verification remains with the main task.